

Sorting Project

PHD Sorters:

Jack, Jeremy, Andrew, Alex,
Vaughn

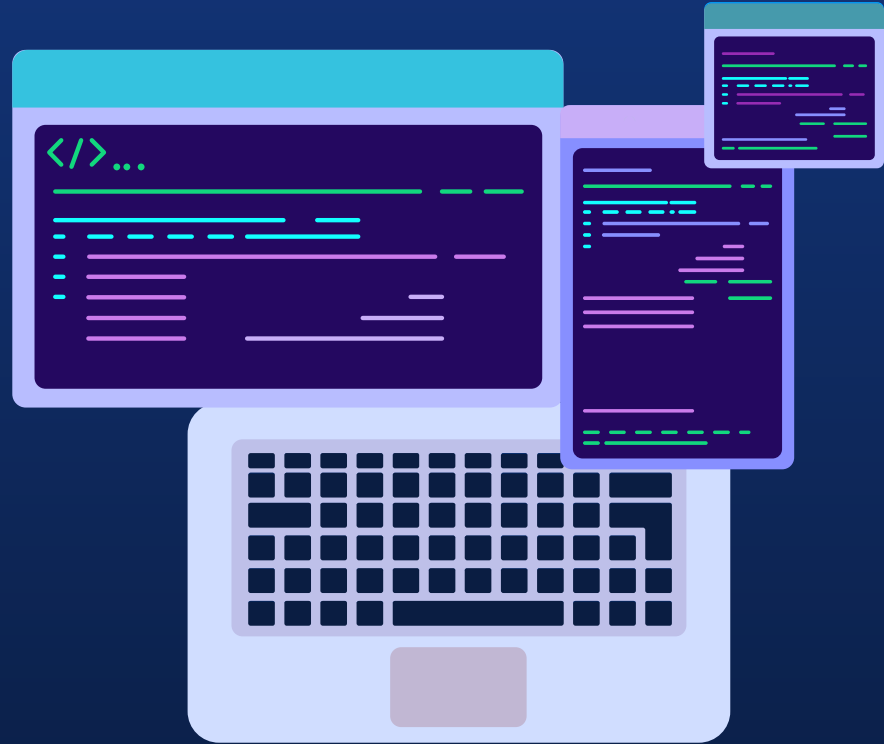


TABLE OF CONTENTS

01

Plan

02

Methods

03

Results

04

Conclusion





01

Plan





General Plan

- Use lab 2 as the basis for the program (since it already had exchange and selection sort)
- Each group member was then responsible for 1 of the 5 remaining sorts
- Once finished, put them all into one file, and then implement counters and timing
- One finished testing and gathering data, use python to graph out the data and compare results





02

Methods & Algorithms





Random Arrays

```
int* generateAlmostSorted(int n) {
    int* arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = i;
    if (n > 5) {
        swap(arr[n / 2], arr[n / 2 - 1]);
        swap(arr[n / 3], arr[n / 3 - 1]);
    }
    return arr;
}

int* generateRandom(int n) {
    int* arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = rand() % n;
    return arr;
}

int* generateReverseSorted(int n) {
    int* arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = n - i;
    return arr;
}

int* generateDuplicates(int n) {
    int* arr = new int[n];
    int vals[5] = {0, n / 4, n / 2, 3 * n / 4, n};
    for (int i = 0; i < n; i++) arr[i] = vals[rand() % 5];
    return arr;
}
```

Random Arrays Cont.

```
int sizes[3] = {1000, 10000, 100000};

for (int s = 0; s < 3; s++) {
    int n = sizes[s];
    cout << "n=====\\n";
    cout << "Array Size = " << n << "\\n";
    cout << "=====\\n";

    int* almostSorted = generateAlmostSorted(n);
    int* randomArr = generateRandom(n);
    int* reverseSorted = generateReverseSorted(n);
    int* duplicates = generateDuplicates(n);

    runSortsOnArray("Almost Sorted", almostSorted, n);
    runSortsOnArray("Random", randomArr, n);
    runSortsOnArray("Reverse Sorted", reverseSorted, n);
    runSortsOnArray("Duplicates", duplicates, n);

    delete[] almostSorted;
    delete[] randomArr;
    delete[] reverseSorted;
    delete[] duplicates;
}
```

Quick Sort Code:

```
// --- Quicksort ---
{
    int quicksortCount = 0, partitionCount = 0;
    int totalCount = 0, compareCount = 0, swapCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    quicksort(tempArray, 0, size - 1,
              quicksortCount, partitionCount,
              totalCount, compareCount, swapCount);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Quicksort - " << name << " (n=" << size << ")\n";
    cout << "  Quicksort Calls: " << quicksortCount << "\n";
    cout << "  Partition Calls: " << partitionCount << "\n";
    cout << "  Total Calls: " << totalCount << "\n";
    cout << "  Compares: " << compareCount << "\n";
    cout << "  Swaps: " << swapCount << "\n";
    cout << "  Time (us): " << (end - start) << "\n\n";

    delete[] tempArray;
}
```

Int (main)

```
// Quicksort
void partition(int arr[], int low, int high, int& pivotpoint,
              int& partition_count, int& total_count,
              int& compare_count, int& swap_count) {
    partition_count++;
    total_count++;

    int pivotitem = arr[low];
    int j = low;

    for (int i = low + 1; i <= high; ++i) {
        compare_count++;
        if (arr[i] < pivotitem) {
            j++;
            int tmp = arr[i];
            arr[i] = arr[j];
            arr[j] = tmp;
            swap_count++;
        }
    }

    pivotpoint = j;
    int tmp = arr[low];
    arr[low] = arr[pivotpoint];
    arr[pivotpoint] = tmp;
    swap_count++;
}

void quicksort(int arr[], int low, int high,
              int& quicksort_count, int& partition_count,
              int& total_count, int& compare_count, int& swap_count) {
    if (low >= high) return;

    int pivotIndex = low + rand() % (high - low + 1);
    swap(arr[low], arr[pivotIndex]);
    swap_count++;

    quicksort_count++;
    total_count++;

    int pivotpoint = -1;
    partition(arr, low, high, pivotpoint,
              partition_count, total_count,
              compare_count, swap_count);

    quicksort(arr, low, pivotpoint - 1,
              quicksort_count, partition_count,
              total_count, compare_count, swap_count);
    quicksort(arr, pivotpoint + 1, high,
              quicksort_count, partition_count,
              total_count, compare_count, swap_count);
}

// Array Generation
```


Selection Sort Code:

```
// --- Selection Sort ---
{
    int compCount = 0, swapCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    selectionSort(tempArray, size, compCount, swapCount);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Selection Sort - " << name << " (n=" << size << ")\\n";
    cout << "   Compares: " << compCount << "\\n";
    cout << "   Swaps: " << swapCount << "\\n";
    cout << "   Time (us): " << (end - start) << "\\n\\n";

    delete[] tempArray;
}
```

Int (main)

```
// Selection Sort
void selectionSort(int myNumbers[], int n, int& comp_count, int& swap_count) {
    int smallest;
    for (int i = 0; i < n - 1; i++) {
        smallest = i;
        for (int j = i + 1; j < n; j++) {
            comp_count++;
            if (myNumbers[j] < myNumbers[smallest]) {
                smallest = j;
            }
        }
        if (smallest != i) {
            int temp = myNumbers[smallest];
            myNumbers[smallest] = myNumbers[i];
            myNumbers[i] = temp;
            swap_count++;
        }
    }
}
```

Exchange Sort Code:

```
// Exchange Sort
void exchangeSort(int myNumbers[], int n, int& comp_count, int& swap_count) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            comp_count++;
            if (myNumbers[j] < myNumbers[i]) {
                int temp = myNumbers[i];
                myNumbers[i] = myNumbers[j];
                myNumbers[j] = temp;
                swap_count++;
            }
        }
    }
}
```

```
// --- Exchange Sort ---
{
    int compCount = 0, swapCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    exchangeSort(tempArray, size, compCount, swapCount);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Exchange Sort - " << name << " (n=" << size << ")\n";
    cout << "  Compares: " << compCount << "\n";
    cout << "  Swaps: " << swapCount << "\n";
    cout << "  Time (us): " << (end - start) << "\n\n";

    delete[] tempArray;
}
```

Int (main)

Merge Sort Code:

```
// --- Merge Sort ---
{
    mergeCompCount = 0;
    mergeAssignCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    mergeSort(tempArray, 0, size - 1);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Merge Sort - " << name << " (n=" << size << ")\n";
    cout << " Comparisons: " << mergeCompCount << "\n";
    cout << " Assignments: " << mergeAssignCount << "\n";
    cout << " Time (us): " << (end - start) << "\n\n";

    delete[] tempArray;
}
```

Int (main)

```
// Merge sort
int mergeCompCount;
int mergeAssignCount;

void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
        mergeAssignCount++;
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[mid + 1 + j];
        mergeAssignCount++;
    }

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        mergeCompCount++;
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
            mergeAssignCount++;
        } else {
            arr[k++] = R[j++];
            mergeAssignCount++;
        }
    }

    while (i < n1) {
        arr[k++] = L[i++];
        mergeAssignCount++;
    }
    while (j < n2) {
        arr[k++] = R[j++];
        mergeAssignCount++;
    }

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Heap Sort Code:

```
// --- Heap Sort ---
{
    int compCount = 0, swapCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    heapSort(tempArray, size, compCount, swapCount);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Heap Sort - " << name << " (n=" << size << ")\n";
    cout << " Comparisons: " << compCount << "\n";
    cout << " Assignments: " << swapCount << "\n";
    cout << " Time (us): " << (end - start) << "\n\n";

    delete[] tempArray;
}
```

Int (main)

```
// Heap Sort
void heapify(int arr[], int n, int i, int& comp_count, int& swap_count) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;
    if (l < n) {
        comp_count++;
        if (arr[l] > arr[largest]) largest = l;
    }
    if (r < n) {
        comp_count++;
        if (arr[r] > arr[largest]) largest = r;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        swap_count++;
        heapify(arr, n, largest, comp_count, swap_count);
    }
}

void heapSort(int arr[], int n, int& comp_count, int& swap_count) {
    for (int i = n / 2 - 1; i >= 0; --i)
        heapify(arr, n, i, comp_count, swap_count);
    for (int i = n - 1; i >= 0; --i) {
        swap(arr[0], arr[i]);
        swap_count++;
        heapify(arr, i, 0, comp_count, swap_count);
    }
}
```

Bubble Sort Code:

```
// --- Bubble Sort ---
{
    int compCount = 0, swapCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    bubbleSort(tempArray, size, compCount, swapCount);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Bubble Sort - " << name << " (n=" << size << ")\n";
    cout << "  Compares: " << compCount << "\n";
    cout << "  Swaps: " << swapCount << "\n";
    cout << "  Time (us): " << (end - start) << "\n\n";

    delete[] tempArray;
}
```

Int (main)

```
//bubble sort
void bubbleSort(int arr[], int n, int& comp_count, int& swap_count){
    bool notSorted = true;
    int i, j, temp;

    while (notSorted){
        for (i = 0; i < n; i++){
            notSorted = false;
            for (j = 0; j < n-i-1; j++){
                comp_count++;
                if (arr[j] > arr[j + 1]){
                    notSorted = true;
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swap_count++;
                }
            }
        }
    }
}
```

Time Complexity:
Worst: $O(n^2)$, Best: $O(n)$

Insertion Sort Code:

```
// Insertion Sort
void insertionSort(int arr[], int n, long long& comp_count, long long& swap_count) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0) {
            comp_count++;
            if (arr[j] > key) {
                arr[j + 1] = arr[j];
                swap_count++;
                j--;
            } else {
                break;
            }
        }
        arr[j + 1] = key;
    }
}
```

```
// --- Insertion Sort ---
{
    long long compCount = 0, swapCount = 0;
    int* tempArray = new int[size];
    copy(arr, arr + size, tempArray);

    start = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    insertionSort(tempArray, size, compCount, swapCount);
    end = duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();

    cout << "Insertion Sort - " << name << " (n=" << size << ")\n";
    cout << "   Compares: " << compCount << "\n";
    cout << "   Swaps/Shifts: " << swapCount << "\n";
    cout << "   Time (us): " << (end - start) << "\n\n";

    delete[] tempArray;
}
```

Int (main)



Gathering Data

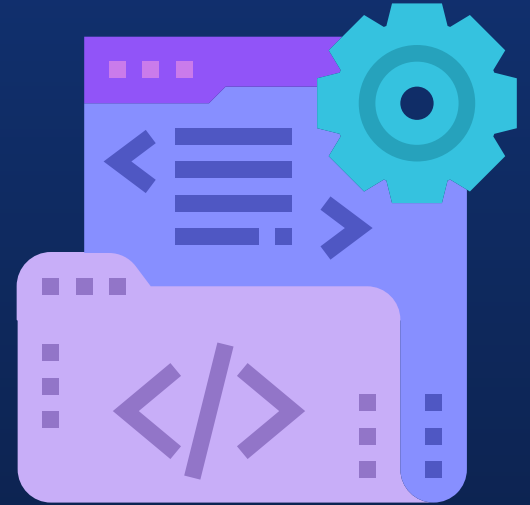
- Creates random arrays for $n = 1000, 10000, \& 10000$ that are randomly sorted, almost sorted, reverse sorted, & multiple duplicates.
- With each execution, we tracked compares and time, in milliseconds, (and other counts when applicable)
- Ran the program ? times to generate (basically ? times for each of the 3 n values and each of the 4 different types of arrays) tons of data and then used python code to create graphs.





03

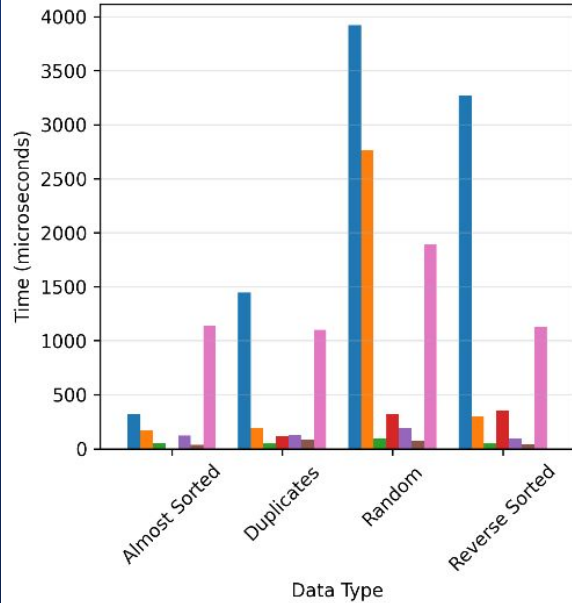
Testing Results



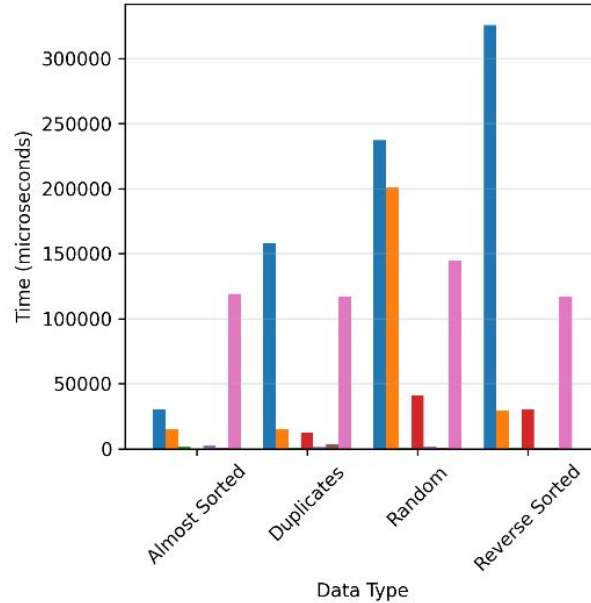
Sorting Time By Array Size:

- Bubble Sort
- Exchange Sort
- Heap Sort
- Insertion Sort
- Merge Sort
- Quicksort
- Selection Sort

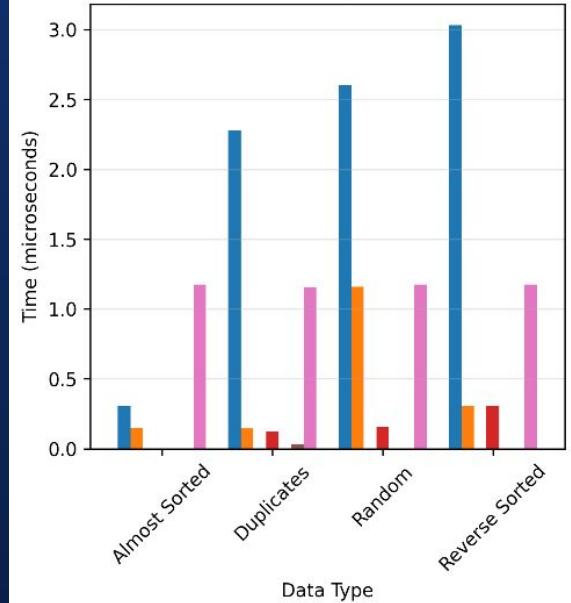
Execution Time - Array Size 1000



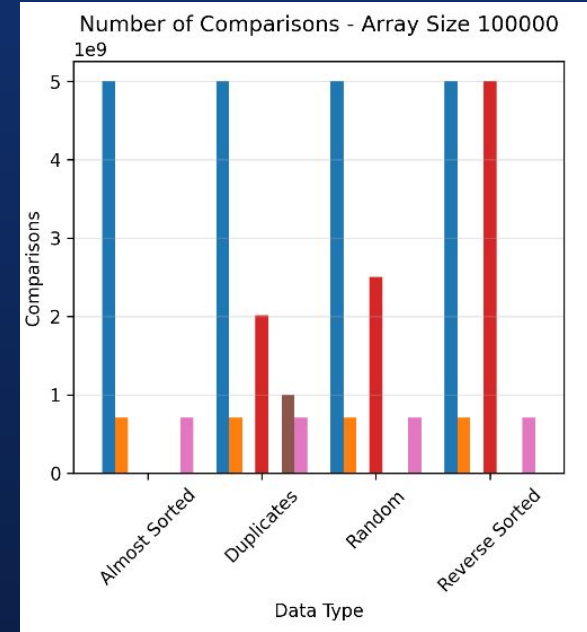
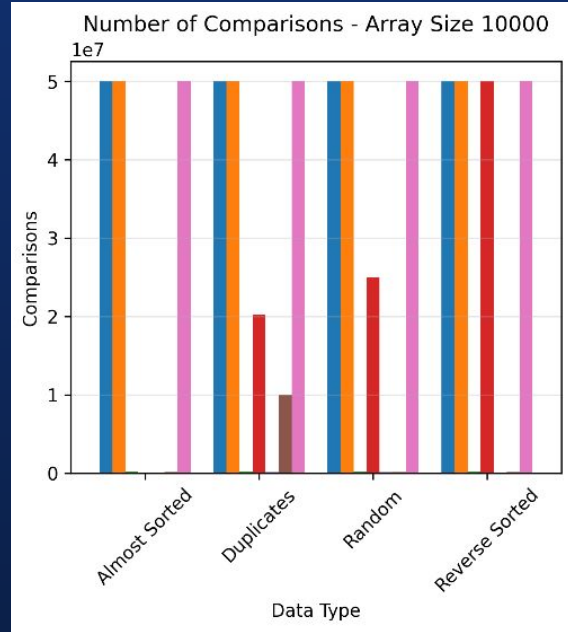
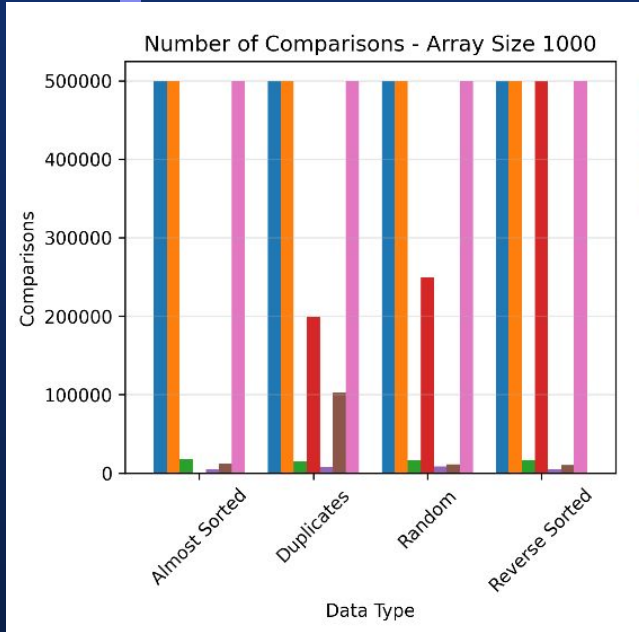
Execution Time - Array Size 10000



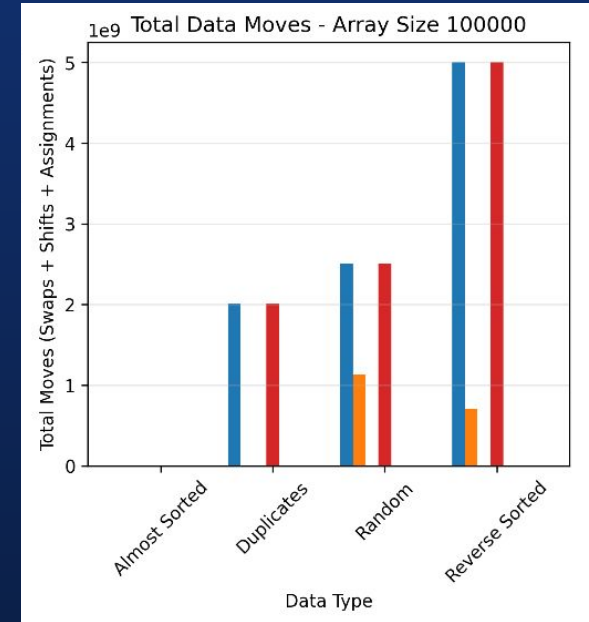
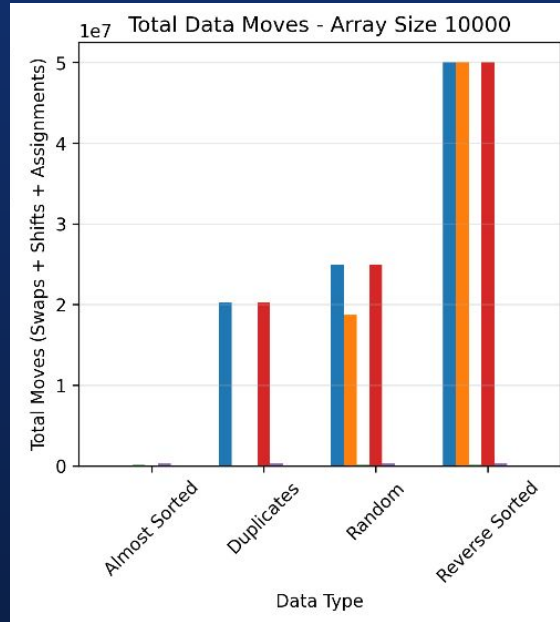
1e7 Execution Time - Array Size 100000



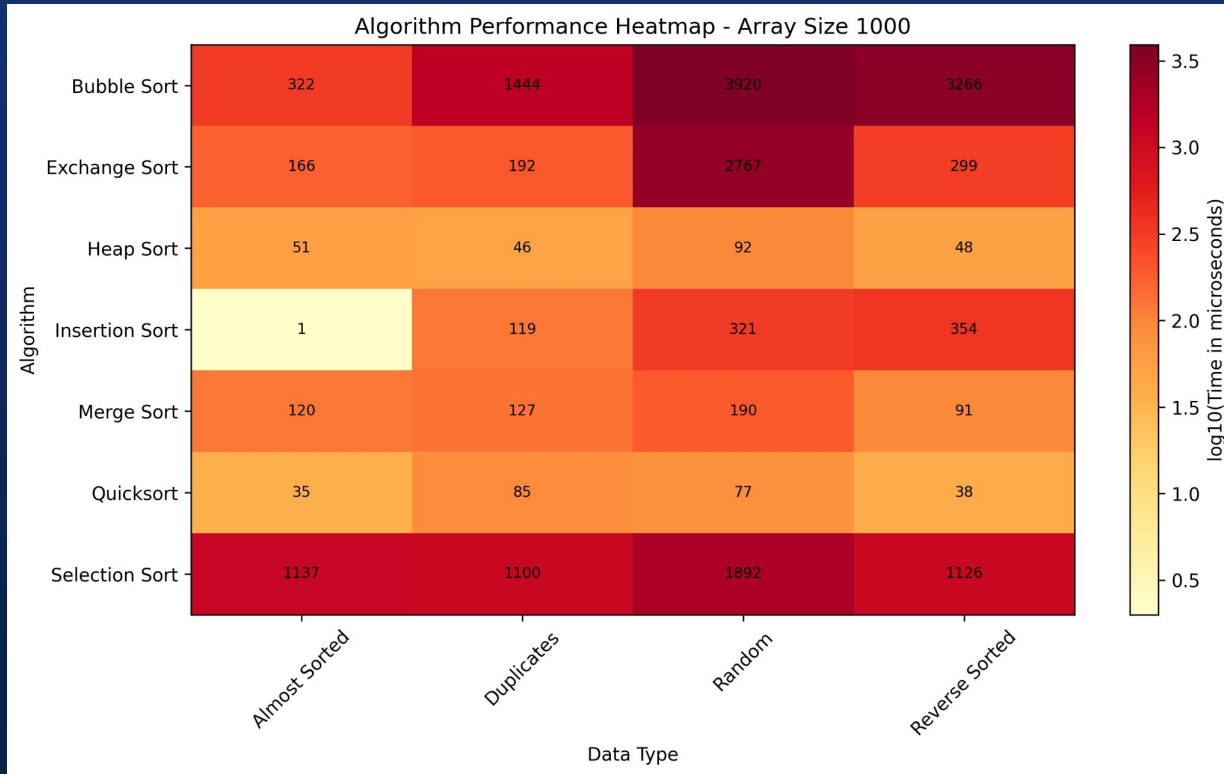
Number of Comparisons By Array Size:



Number of Moves By Array Size:



Heat Map of Algorithms on Array Size 1000:





04

Conclusion





Overcoming challenges

- Updated swap_count and comp_count from int to long long in both insertion sort and bubble sort to avoid integer overflow when tracking operations on large input sizes.
- Re-uploaded updated code to Google Drive and communicated regularly to ensure everyone worked with the latest version.
- Keeping the updated code file organized due to the amount of sorts and information
- Finding a the best way to represent our findings that efficiently shows certain sorts superiority over others when testing specific array types





Analysis

Small Inputs ($n = 1,000$):

- All algorithms complete quickly, but inefficient ones (bubble, exchange, selection) already show higher comparisons.

Medium Inputs ($n = 10,000$):

- Clear performance split: $O(n^2)$ sorts (bubble, exchange, selection, insertion) become much slower than $O(n \log n)$ sorts (merge, quicksort, heap).

Large Inputs ($n = 100,000$):

- $O(n^2)$ sorts are essentially impractical (huge time/operations).
- Quicksort is generally fastest, except when unlucky with pivots (we used a random pivot to help its overall performance)
- Merge sort and heapsort were consistently reliable and reasonably efficient.



Conclusion

- Trade-offs:
 - Simple algorithms are easier to code but scale poorly.
- Efficiency:
 - Quicksort is fastest in most cases, merge sort more predictable, heapsort a good middle ground.
- Data Structure Matters:
 - Input order heavily influences runtime